

Habilidades esenciales en AI Engineering

Lo que todo desarrollador necesita saber para construir con IA hoy.

Origen del mapa

10.000+

ofertas de empleo analizadas para identificar las habilidades más demandadas.

02

Decenas de entrevistas con expertos, líderes de contratación y reclutadores

03

Encuestas y síntesis de datos públicos

Objetivo

Ayudar a priorizar qué aprender y a quién contratar

Habilidad, no solo un rol

ROL

AI Engineer

Persona cuyo trabajo es construir sistemas de IA. Un grupo acotado.

HABILIDAD

AI Engineering

Conjunto de capacidades que necesitará todo desarrollador.

Full-stack

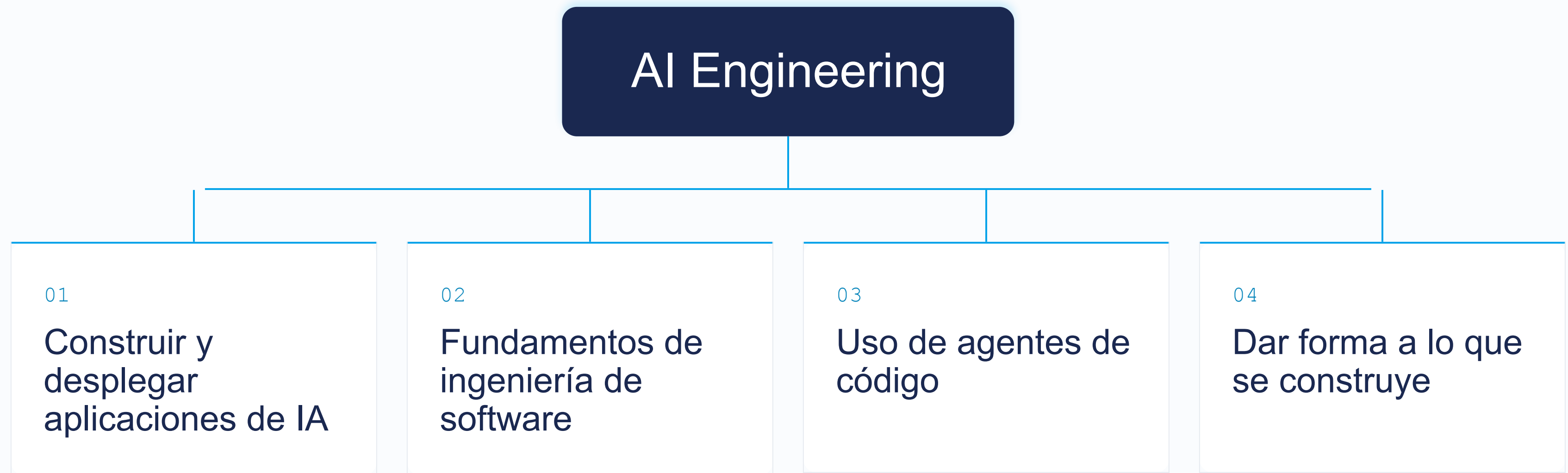
Datos

DevOps

Machine learning

IA

Mapa de habilidades



01

Construir y desplegar aplicaciones de IA

Crear sistemas confiables a partir de componentes cuyas salidas no son predecibles.

Seis competencias de aplicaciones de IA

Fundamentos de LLM

Tokens, ventana de contexto, parámetros de muestreo y uso de herramientas.

Fundamentar con datos

RAG, grafos de conocimiento y capas semánticas para dar buen contexto.

Sistemas agénticos

Flujos, herramientas, memoria, multiagente y guardrails de seguridad.

Desarrollo guiado por evals

Medir con evaluaciones y análisis de errores para decidir qué probar.

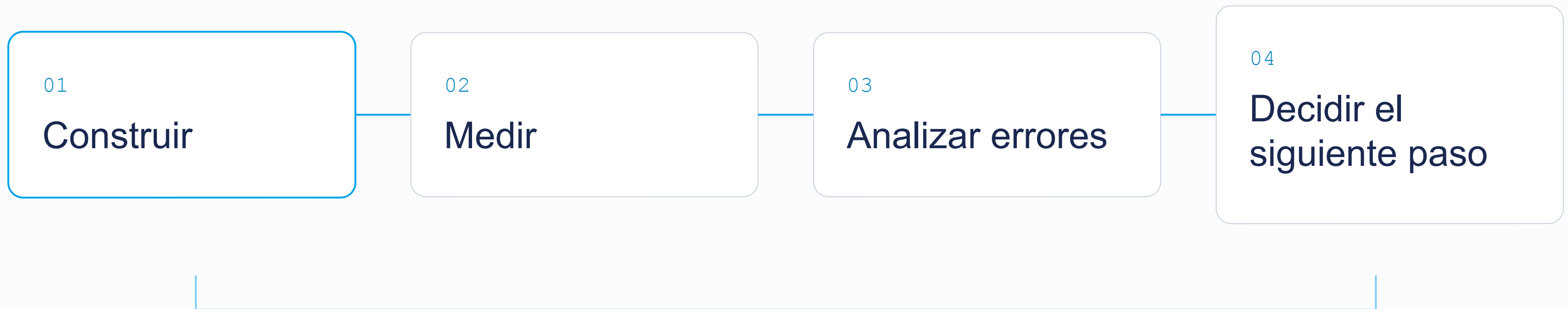
Operación en producción

Observabilidad, detección de drift, costo y latencia a escala.

Fundamentos de ML

Aprendizaje supervisado, sesgo/varianza e ingeniería de datos.

Ciclo de evaluación y análisis de errores



Determinista

Reglas en código para salidas verificables.

LLM-as-a-judge

Un modelo califica según criterios definidos.

Humano en el ciclo

Revisión experta donde el juicio es crítico.

02

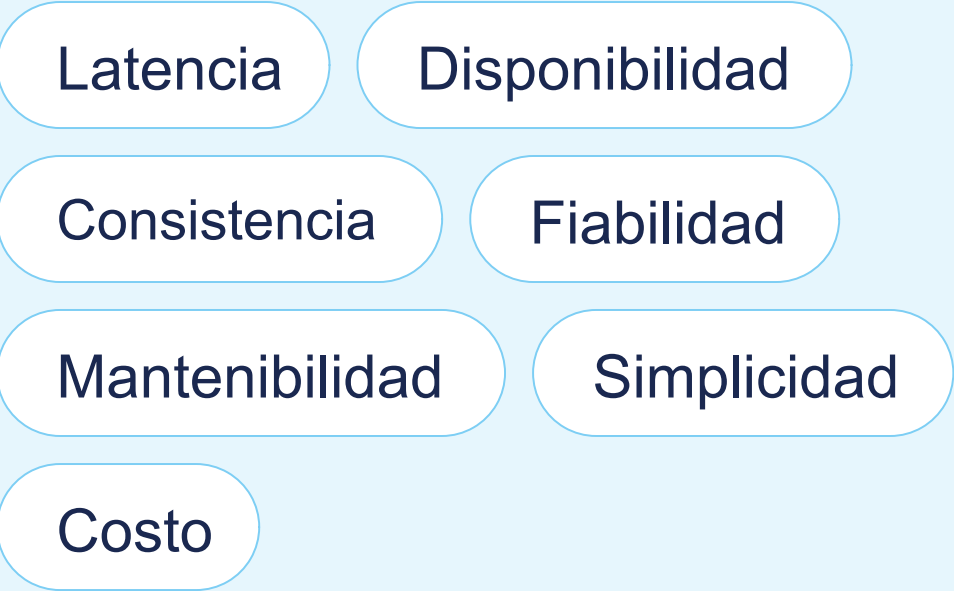
Fundamentos de ingeniería de software

Entender cómo funciona el software para dirigir al agente con el lenguaje preciso de la ingeniería.

Cinco competencias y sus compensaciones

- 01 Aplicaciones full-stack
- 02 Gestión de datos
- 03 Arquitectura de sistemas
- 04 Seguridad y fiabilidad
- 05 Escala y operación en producción

COMPENSACIONES A DIRIGIR



Si no sabe que existen, el agente las decide por usted.

03

Uso de agentes de código

Dirigir agentes —cuánto intervenir y cuánto delegar— para construir software robusto sin desperdiciar tiempo ni tokens.

Flujo de trabajo con agentes

● Fase 01

Planificar

Investigar y entender el código existente. Escribir la spec: requisitos, diseño y arquitectura. Generar y revisar el plan.

● Fase 02

Ejecutar

El agente construye con autonomía calibrada. Verificar con pruebas automáticas y revisión humana.

● Fase 03

Desplegar y monitorear

Desplegar con CI/CD y controles humanos. Agentes que vigilan logs y proponen mejoras.

Cinco competencias con agentes

01	Dirigir el flujo	Cuánto esfuerzo humano y de agente en cada fase, y cuándo volver atrás.
02	Habilitar autonomía	Nivel de delegación, gestión de contexto, agentes en paralelo y permisos.
03	Revisar el trabajo	Pruebas funcionales y de comportamiento, revisión agéntica y auditorías.
04	Personalizar el entorno	Skills, MCP, hooks y contexto permanente como AGENTS.md.
05	Fundamentos del agente	Cómo un harness envuelve al LLM: búsqueda, contexto y subagentes.

04

Dar forma a lo que se construye

Decidir qué debe construirse, no solo implementar lo que otro especificó.

Cuatro competencias para dar forma

01

Conducir el ciclo de construcción

Prototipo, MVP o sistema empresarial: decidir el siguiente paso con sesgo a la acción.

02

Tomar decisiones de producto

Sentido de producto, diseño y negocio, basado en empatía con el usuario.

03

Comunicar y liderar

Alinear marketing, finanzas y legal; explicar qué es técnicamente viable.

04

Propiedad con alta agencia

Detectar oportunidades, ejecutar de punta a punta y medir por el valor creado.

Glosario de conceptos clave

LLM	Modelo que genera texto token a token a partir de un contexto.	Contexto	Información que el modelo recibe en su ventana para responder.
RAG	Recuperar datos relevantes e incluirlos en el prompt.	Agente	LLM dentro de un harness que decide su siguiente paso y usa herramientas.
MCP	Protocolo abierto para conectar modelos con herramientas y datos.	Eval	Prueba sistemática que mide la calidad de las salidas.
Spec	Documento de requisitos, diseño y arquitectura que guía al agente.	Drift	Degradación del rendimiento cuando cambian los datos reales.
Shift left	Llevar seguridad y pruebas al inicio del ciclo de desarrollo.	MVP	Versión mínima para validar valor con usuarios reales.

Aprendizaje continuo

Seguir la frontera tecnológica, probar herramientas nuevas y ajustar los flujos de trabajo a medida que cambian las buenas prácticas.

Innova Copilot

Las competencias del mapa, juntas en un proyecto real

Un copiloto de IA para un centro de innovación ficticio que atiende MIPYMES de Bolívar. Construido paso a paso, con 11 versiones que puedes revisar en [GitHub](#).

Del problema a una propuesta, en una conversación

PROCESO MANUAL

Hoy

Una empresa escribe; alguien del centro la entrevista, encuadra el caso, hace el análisis a mano y arma la oferta. Semanas.

INNOVA COPILOT

La idea

Diagnóstico → línea de trabajo → servicio del catálogo con cita → simulación "what if" → pre-propuesta.



Ilustración generada con IA (Gemini)

¿Por qué no solo IA? El LLM entiende y redacta; los precios salen del catálogo, los cálculos de una herramienta y la decisión es de una persona.

EL CASO DE LA DEMO

Hotel Brisas de Getsemaní

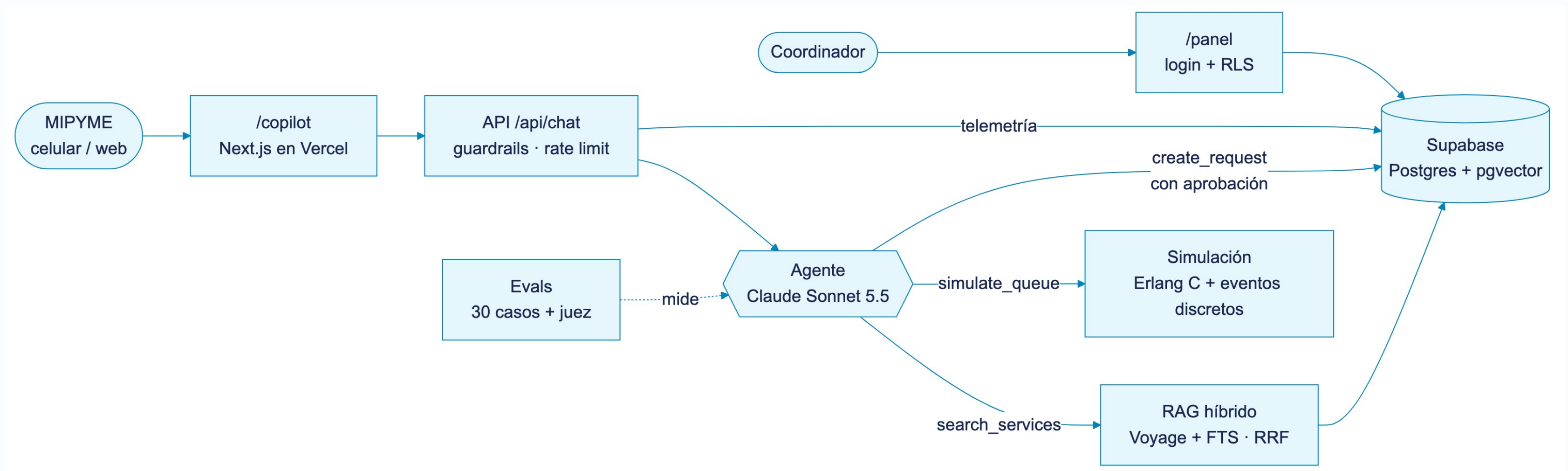
24 habitaciones · Cartagena
Filas de hasta 40 min en el check-in
2 recepcionistas
18 huéspedes por hora en el pico
6 minutos por check-in
Quejas en las reseñas

11 pasos, 11 tags de git



`git checkout paso-05-rag` → el proyecto exactamente como estaba en ese punto, funcionando. Cada paso tiene su doc, su bitácora del agente y su "reprodúcelo tú".

Arquitectura simple, en producción desde el día 1



Next.js + Vercel

UI y API juntas; cada push despliega

Supabase

Postgres, vectores, auth y RLS en todas las tablas

Claude Sonnet 5.5

agente; Haiku 4.5 como juez de las evals

Sin datos, el modelo inventa con seguridad

Pre-propuesta

- **Problema:** Filas de hasta 40 minutos en check-in (2 a 5 p. m.) en temporada alta, con quejas en reseñas.
- **Línea de trabajo:** process_design
- **Servicios:** Rediseño de procesos de atención + digitalización del check-in.
- **Alcance sugerido:**
 - Observación y medición de tiempos en recepción (1 a 2 días).
 - Mapa del proceso actual y propuesta del nuevo.
 - Implementación del pre-check-in digital.
 - Capacitación al equipo y plan de seguimiento.
 - Idealmente, listo antes de diciembre.
- **Inversión aproximada:** entre **COP 6 y 12 millones**, según el nivel de digitalización (valor de referencia de la demo, no una cotización).
- **Siguiente paso:** Envíame el **nombre de la persona de contacto** y confirma el nombre de la empresa (Hotel Brisas de Getsemani) para dejar registrada la pre-propuesta y que un asesor del centro agende una visita de diagnóstico.

Respuesta real del paso 03 (docs/evidencia/paso-03)

COP 6–12 M

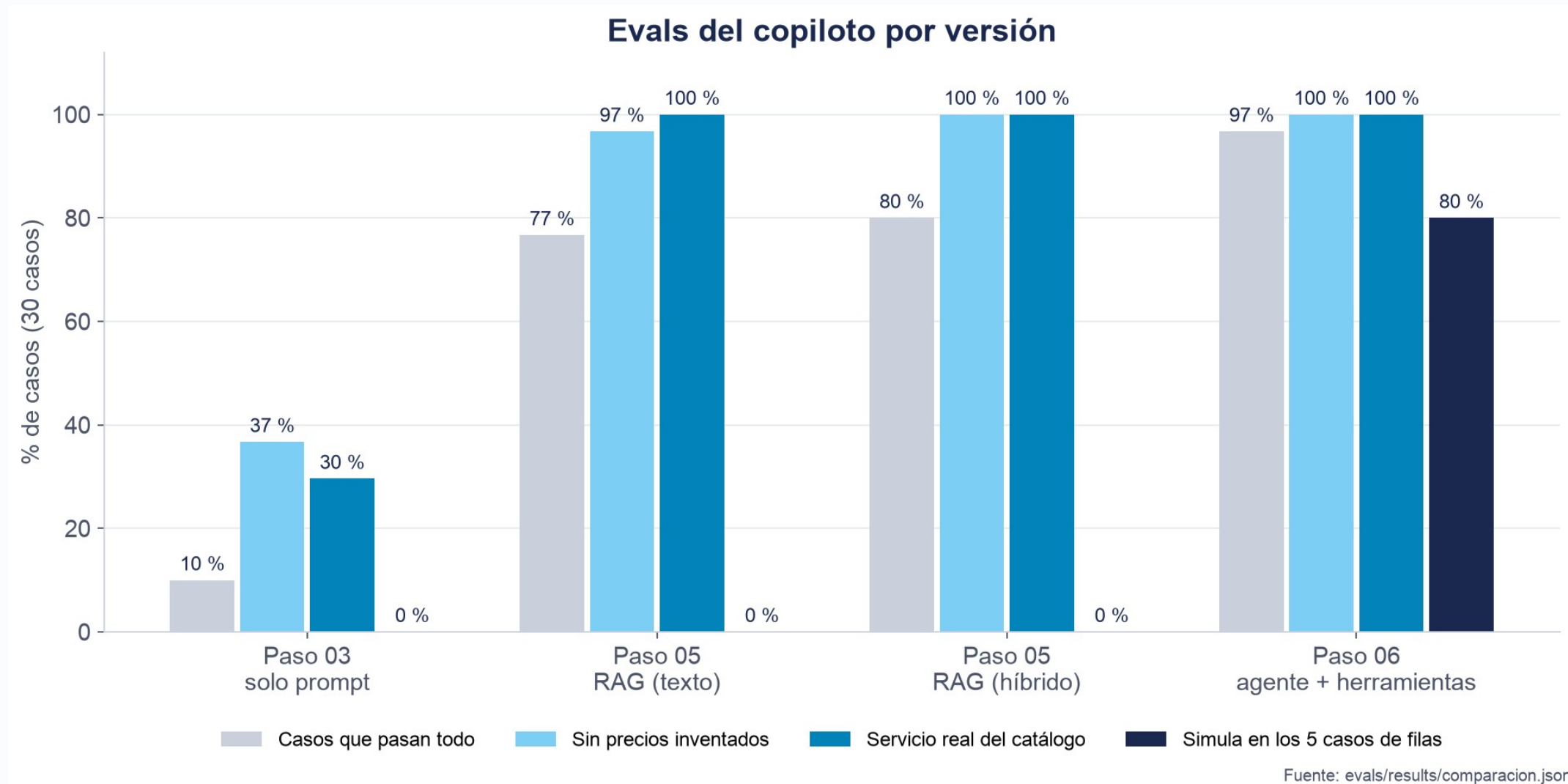
rango que "recomendó" el modelo para el hotel...

COP 8–25 M

...y lo que dice la ficha real del catálogo

Tokens, contexto, caché: el caché bajó un 78,6 % el costo de la entrada. `temperature` ya no existe en Sonnet 5.5: el control es `effort`.

Sin evals, cada cambio es una opinión



3 → 24 → 29

de 30 casos pasan todos los chequeos (pasos 03 → 05 → 06)

37 % → 100 %

de respuestas sin precios inventados

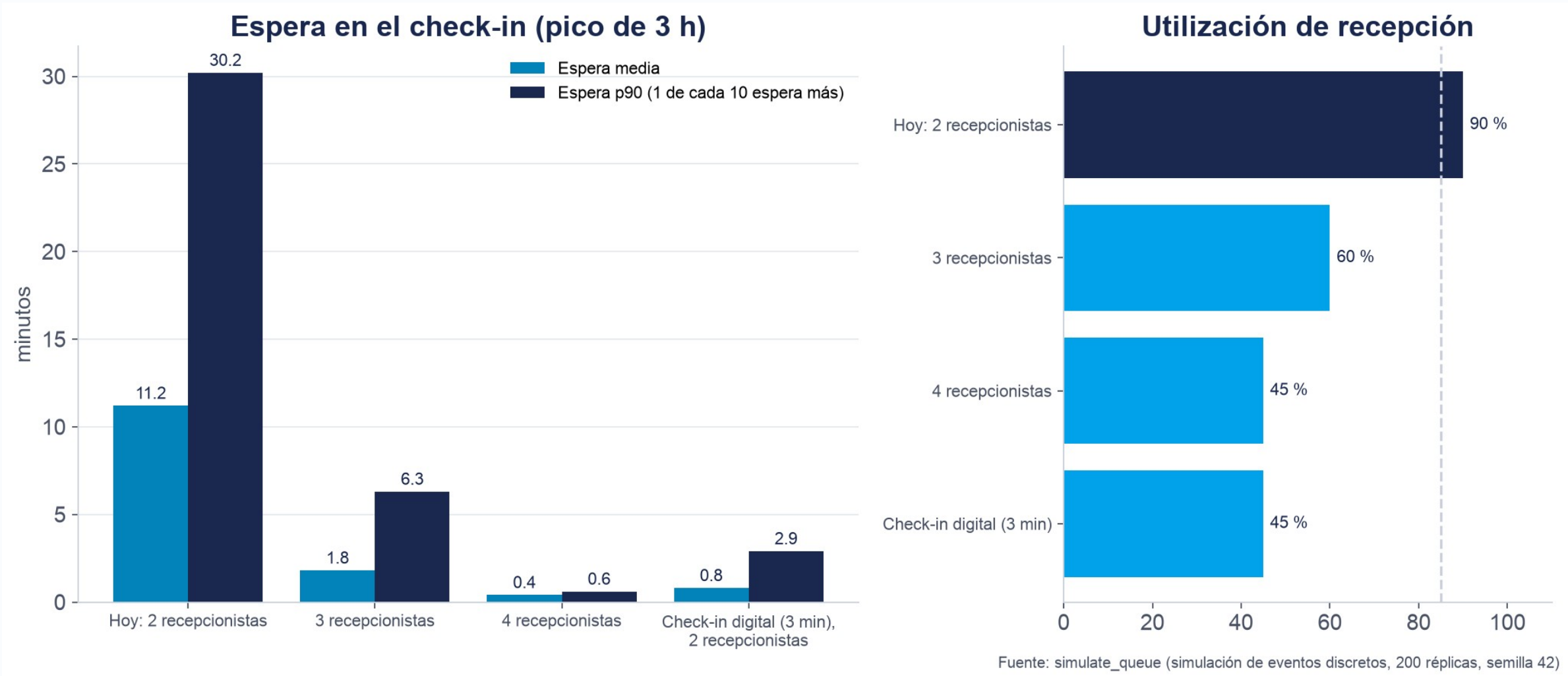
La eval también tuvo bugs: el primer juez premió un precio inventado y el primer dataset castigaba hacer preguntas. Lo vimos leyendo los fallos uno por uno.

No cambiamos el modelo: cambiamos lo que sabe

SERVICIO CORRECTO 30 % → 100 % recomienda un servicio real del catálogo, con enlace a la ficha	BÚSQUEDA FTS 6/8 · híbrido 8/8 con paráfrasis sin palabras del catálogo ("botamos comida todos los días")	TÉCNICA Small-to-big busca en fragmentos, entrega al modelo la ficha completa: nunca un precio suelto
---	--	--

En el prompt (con caché)	reglas · 3 políticas · índice de nombres de servicios, sin precios
Por recuperación	fichas completas: precios, entregables, duración
Ni grafo ni base vectorial aparte	13 documentos sin relaciones: Postgres + pgvector basta

El LLM no calcula: decide quién calcula



`simulate_queue`: Erlang C + simulación de eventos discretos con semilla fija, 17 tests. El texto del agente usó exactamente sus cifras en 4 de 4 casos de la eval.

30,2 min

espera p90 hoy (1 de cada 10 huéspedes espera más)

0,8 min

espera media con check-in digital previo (con una 3.^a recepcionista: 1,8 min)

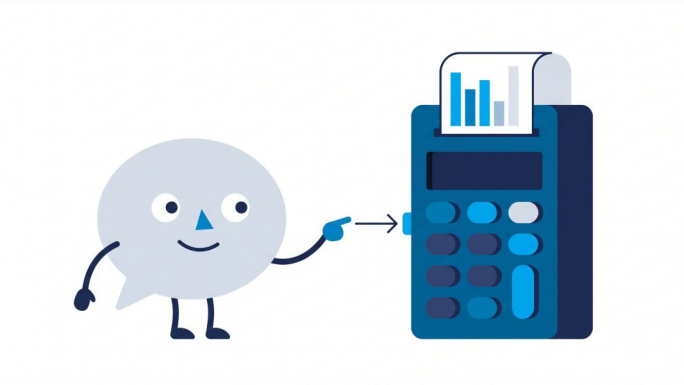


Ilustración generada con IA (Gemini)

Actuar, pero con permiso

Pre-propuesta · Hotel Brisas de Getsemaní

Guardada

Problema

Hotel boutique de 24 habitaciones en Getsemaní. En temporada alta (diciembre y enero) hay filas de hasta 40 minutos en el check-in entre 2 y 5 p. m. Tienen 2 recepcionistas, unas 18 llegadas por hora en el pico y unos 6 minutos por check-in. Hay quejas en las reseñas.

Línea de trabajo

Diseño, mejora y sostenibilidad de procesos

Servicios

simulacion-operaciones, digitalizacion-procesos-administrativos

Alcance sugerido

Validar con datos reales las llegadas y los tiempos de check-in. Comparar escenarios what-if de 2, 3 y 4 recepcionistas y de check-in digital previo. Si conviene, diseñar y digitalizar el proceso de pre check-in. Descuento MIPYME por confirmar según el número de empleados.

Inversión (según catálogo)

Modelado y digitalización de procesos administrativos: COP 5.000.000 – 18.000.000 · Simulación de operaciones y análisis "what if": COP 8.000.000 – 25.000.000

Siguiente paso

Un coordinador del centro revisa la solicitud y te contacta en unos 2 días hábiles.

Contacto: Marcela Pérez

Aprobación humana

Guardar la pre-propuesta exige el botón "Sí, guardar", y la aprobación va firmada.

El precio lo pone el código

El rango de inversión sale del catálogo, no del modelo.

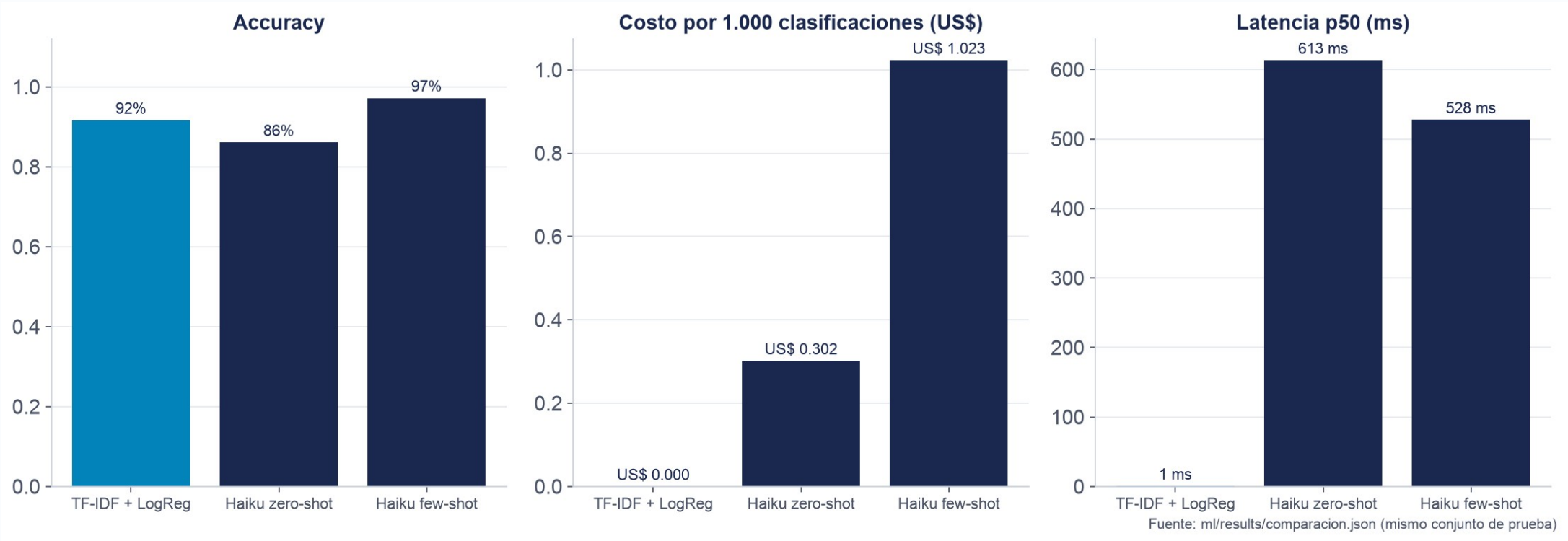
Protección de la URL del QR

Límite de mensajes por IP y sesión, tope diario, límite de tamaño y fallo cerrado.

Plan B

AI_MODE=mock: la misma demo sin red ni claves.

¿Siempre un LLM? Mismo test, tres modelos



Modelo	Accuracy	Costo / 1.000	Latencia p50
TF-IDF + regresión logística	91,7 %	≈ US\$ 0 (local)	0,6 ms
Haiku 4.5 zero-shot	86,1 %	US\$ 0,30	613,3 ms
Haiku 4.5 few-shot	97,2 %	US\$ 1,02	527,7 ms

36 casos de prueba sintéticos: 1 caso = 2,8 puntos. Decisión: el agente clasifica en la conversación; el modelo clásico, para triage masivo y drift.

Todo esto lo construyó un agente de código

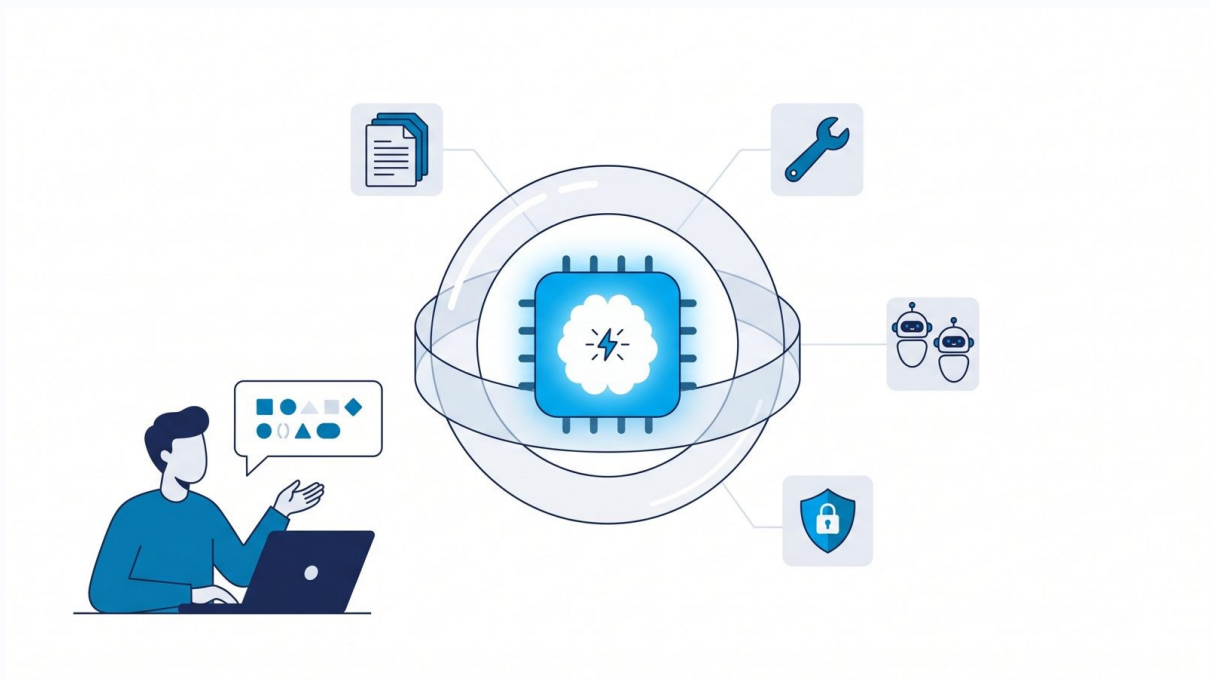


Ilustración generada con IA (Gemini)

CLAUDE.md	la inducción del agente: arquitectura, comandos, "nunca inventes métricas"
Hooks y permisos	bloquea escribir claves; no puede leer .env ni forzar un push
Subagente revisor	contexto limpio, solo lectura: "NOT READY", 3 fallas graves en 7 minutos
Bitácora	el prompt exacto de cada paso y cada error, sin editar

DE LA BITÁCORA DEL PASO 06

El modelo escribió: "te di cifras de simulación sin haber consultado las herramientas. No debí hacerlo". Sí las había consultado: el bug era nuestro (el historial perdía las llamadas). El modelo "confesando" un error que no cometió fue la pista.

Lo que no se mide en producción, no existe

US\$ 0,016

costo promedio por conversación

6,2 / 13,2 s

latencia por respuesta, p50 / p95

TVD 0,30

alerta de drift (umbral 0,2): avalancha de solicitudes de propiedad intelectual

Telemetría real de 10 conversaciones (15 respuestas) hasta el 2026-09-29: muestra pequeña. El drift usa tráfico simulado y marcado como tal.

Telemetría por turno

tokens, costo, latencia, herramientas, errores

Evals de regresión

un cambio de prompt sin eval en vivo no pasa el CI

Runbook

API caída → modo mock; costo → tope diario; abuso → límites; rollback

Riesgo con fecha

Haiku 4.5 se retira no antes del 15 oct 2026

Una lección por pilar

01 · APPS DE IA

Los datos pesan más que el modelo

El mismo modelo pasó de 10 % a 80 % de casos correctos con RAG. Y las cifras las calcula una herramienta.

03 · AGENTES DE CÓDIGO

Una regla pide; un hook garantiza

Permisos, hooks, CI y un revisor con contexto limpio. El humano decidió qué construir y cuándo recortar.

02 · SOFTWARE

Prueba como atacante, no como usuario

Los guardrails pasaban las pruebas de la UI; el revisor fabricó el historial y encontró tres formas de disparar la factura.

04 · DAR FORMA

Mide antes de creer, incluso tu eval

El primer 53 % era un defecto del dataset. Leer los fallos a mano evitó una conclusión falsa.

Pruébalo y revísalo tú



El código, paso a paso

`github.com/FutureTales/ai-engineering-demo`



La app en vivo

`innova-copilot.vercel.app`

Demo educativa · el centro, sus servicios y las empresas son ficticios

Fuentes

14 ago 2026	Parte 1 — The AI Engineering Skills Map
21 ago 2026	Parte 2 — Building and Deploying AI Applications
28 ago 2026	Parte 3 — Software Engineering Fundamentals
04 sep 2026	Parte 4 — Using Coding Agents
11 sep 2026	Parte 5 — Shaping the Build